



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 10
по курсу «Языки и методы программирования»
«Реализация итераторов на языке C++»

Студент группы ИУ9-22Б Ивенкова О. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Задание

Согласно выбранному из таблицы описанию требуется составить контейнерный класс (или шаблон контейнерного класса) и итератор для перебора содержимого объектов этого класса. Если в варианте задания говорится о константном итераторе, значит итератор не должен поддерживать изменение содержимого объектов контейнерного класса.

1. Кольцевой буфер с двунаправленным итератором по элементам представляемой им очереди (элементы должны перебираться в том порядке, в каком они добавлялись в очередь).

2. Последовательность множеств с константным двунаправленным итератором по пересечениям соседних множеств последовательности. Обращение к элементам последовательности должно осуществляться с помощью перегруженной операции «[]».

2 Результаты

Исходный код программы представлен в листингах 1– 2.

Листинг 1: Реализация решения первой задачи

```
1 #include <iostream>
2 #include <vector>
3 #include <iterator>
4
5 using namespace std;
6
7 template<typename T>
8 class CircularBuffer {
9 public:
10     class Iterator;
11
12     CircularBuffer(size_t capacity)
13         : capacity(capacity), start(0), endIndex(0), size(0), buffer(
14             capacity) {}
15
16     void add(const T& item) {
17         buffer[endIndex] = item;
18         endIndex = (endIndex + 1) % capacity;
19         if (size == capacity) {
20             start = (start + 1) % capacity;
```

```

20         } else {
21             ++size;
22         }
23     }
24
25     ~CircularBuffer() {
26         buffer.clear();
27     }
28
29     T& operator [] (size_t index) {
30         return buffer[(start + index) % capacity];
31     }
32
33     size_t getSize() const {
34         return size;
35     }
36
37     Iterator begin() {
38         return Iterator(this, 0);
39     }
40
41     Iterator end() {
42         return Iterator(this, size - 1);
43     }
44
45     class Iterator {
46     public:
47         using iterator_category = std::bidirectional_iterator_tag;
48         using value_type = T;
49         using difference_type = std::ptrdiff_t;
50         using pointer = T*;
51         using reference = T&;
52
53         Iterator(CircularBuffer* buffer, size_t position)
54             : buffer(buffer), position(position) {}
55
56         reference operator*() {
57             return (*buffer)[position];
58         }
59
60         pointer operator->() {
61             return &(*buffer)[position];
62         }
63
64         Iterator& operator++() {
65             if (position == buffer->capacity - 1) {

```

```

66         position = 0;
67     } else {
68         ++position;
69     }
70     return *this;
71 }
72
73 Iterator operator++(int) {
74     Iterator temp = *this;
75     ++(*this);
76     return temp;
77 }
78
79
80 Iterator& operator--() {
81     if (position == 0) {
82         position = buffer->capacity - 1;
83     } else {
84         --position;
85     }
86     return *this;
87 }
88 Iterator operator--(int) {
89     Iterator temp = *this;
90     --(*this);
91     return temp;
92 }
93
94 bool operator==(const Iterator& other) const {
95     return position == other.position && buffer == other.buffer;
96 }
97
98 bool operator!=(const Iterator& other) const {
99     return !(*this == other);
100 }
101
102 private :
103     CircularBuffer* buffer;
104     size_t position;
105 };
106
107 private :
108     size_t capacity;
109     size_t start;
110     size_t endIndex;
111     size_t size;

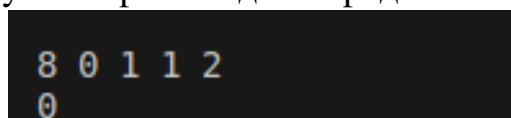
```

```

112     std::vector<T> buffer;
113 };
114
115 int main() {
116     CircularBuffer<int> cb(5);
117
118     cb.add(7);
119     cb.add(8);
120     cb.add(0);
121     cb.add(1);
122     cb.add(1);
123     cb.add(2);
124
125     for (auto it = cb.begin(); it != cb.end(); ++it) {
126         std::cout << *it << " ";
127     }
128     std::cout << *cb.end() << " ";
129     std::cout << std::endl;
130
131     auto it = cb.begin();
132     ++it;
133     ++it;
134     --it;
135     std::cout << *it << std::endl;
136
137     return 0;
138 }

```

Результат запуска первой задачи представлен на рисунке 1.



```

8 0 1 1 2
0

```

Рис. 1 — Результат 1

Листинг 2: Реализация решения второй задачи

```

1 #include <iostream>
2 #include <set>
3 #include <algorithm>
4
5 template<typename T>
6 class SetSequence {
7 private:
8     std::set<T> *sets;
9     size_t size;
10

```

```

11 public :
12     SetSequence(std::set<T> *sets , size_t size) : sets(sets) , size(size)
13     {}
14
15     const std::set<T>& operator [] (size_t index) const {
16         return sets[index];
17     }
18
19     size_t getSize() const {
20         return size;
21     }
22
23     class IntersectionIterator {
24     private :
25         const SetSequence<T> &sequence;
26         size_t current;
27
28     public :
29         IntersectionIterator(const SetSequence<T> &sequence , size_t
30         current) : sequence(sequence) , current(current) {}
31
32         bool operator==(const IntersectionIterator& other) const {
33             return &sequence == &other.sequence && current == other.
34             current;
35         }
36
37         bool operator!=(const IntersectionIterator& other) const {
38             return !(*this == other);
39         }
40
41         IntersectionIterator& operator++() {
42             ++current;
43             return *this;
44         }
45
46         IntersectionIterator& operator--() {
47             --current;
48             return *this;
49         }
50
51         const std::set<T>& operator*() const {
52             static std::set<T> intersection;
53             intersection.clear();
54             if (current < sequence.getSize() - 1) {
55                 std::set_intersection(sequence[current].begin() ,
56                 sequence[current].end() ,

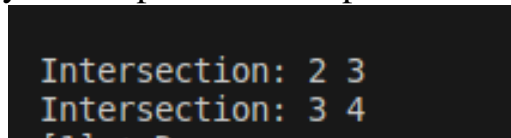
```

```

53         sequence[current + 1].begin(),
sequence[current + 1].end(),
54         std::inserter(intersection,
intersection.begin()));
55     }
56     return intersection;
57 }
58 };
59
60 IntersectionIterator begin() const {
61     return IntersectionIterator(*this, 0);
62 }
63
64 IntersectionIterator end() const {
65     return IntersectionIterator(*this, size - 1);
66 }
67 };
68
69
70
71 int main() {
72     std::set<int> set1 = {1, 2, 3};
73     std::set<int> set2 = {2, 3, 4};
74     std::set<int> set3 = {3, 4, 5};
75
76     SetSequence<int> sequence(&set1, 3);
77
78     for (const auto& intersection : sequence) {
79         std::cout << "Intersection: ";
80         for (const auto& element : intersection) {
81             std::cout << element << " ";
82         }
83         std::cout << std::endl;
84     }
85
86     return 0;
87 }

```

Результат запуска второй задачи представлен на рисунке 2.



```

Intersection: 2 3
Intersection: 3 4

```

Рис. 2 — Результат 2